

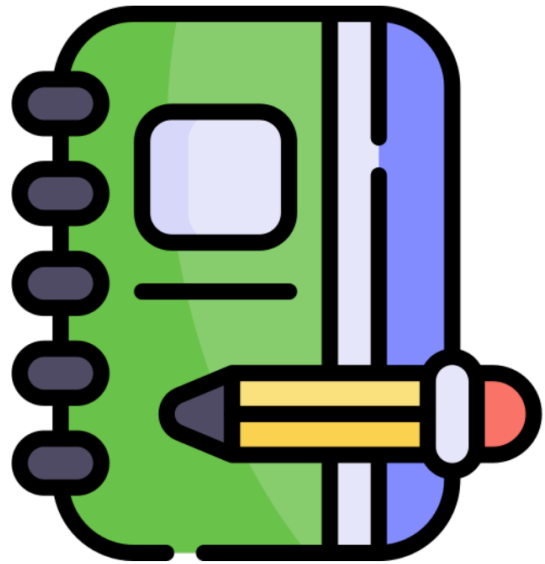
PROGRAMACION 3 - SEGUNDA PARTE

PROGRAMACIÓN DISTRIBUIDA

Programación Distribuida

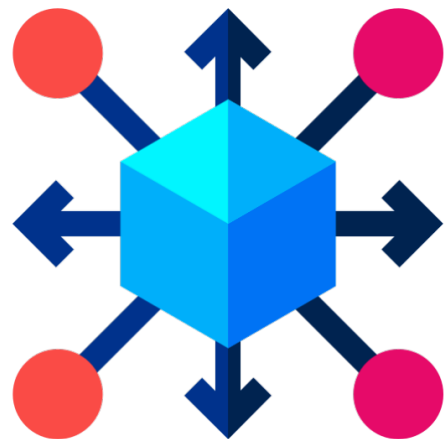
Agenda

- Introducción a la Programación Distribuida
- Conceptos básicos de Comunicación en Red
- Servicios Web
- Servicios Web con SOAP
- Servicios Web con RESTful
- Arquitectura de nuestro proyecto



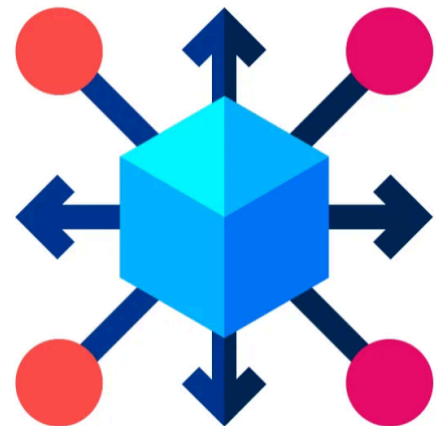
Definición

- Es un paradigma de desarrollo en el cual el software está compuesto por **componentes** que se ejecutan en **múltiples computadoras** que se comunican entre sí a través de una **red**.
- En el contexto moderno, se usa en:
 - Microservicios
 - Aplicaciones móviles/web
 - IoT
 - Servicios en la nube



▼ Características

- **Concurrencia:** múltiples procesos se ejecutan en paralelo.
- **Escalabilidad:** capacidad para crecer eficientemente al aumentar nodos.
- **Transparencia:**
 - De ubicación (no se ve dónde está cada recurso)
 - De acceso (la forma de acceder es uniforme)
 - De concurrencia (manejo automático de múltiples usuarios)
 - De replicación y fallos



▼ Transparencia de ubicación:

Ejemplo Concreto (RPC/RMI)

Un ejemplo concreto de cómo se logra la Transparencia de Ubicación se encuentra en los mecanismos de comunicación como la **Llamada a Procedimiento Remoto (RPC)** o la **Invocación a Métodos Remotos (RMI)**.

El escenario: Imagine que usted está desarrollando un componente de software (el cliente) que necesita obtener el inventario actual de un almacén. Este inventario es un recurso que reside en un sistema de **Back-End** diferente.

Sin Transparencia de Ubicación (Acceso Crudo, Ej. Sockets): El programador tendría que especificar la dirección de red y el puerto exacto del servidor (usando IP y posiblemente TCP), y gestionar el envío y recepción de datos a bajo nivel. El código del cliente estaría inherentemente ligado a la ubicación física del servidor.

Con Transparencia de Ubicación (Ejemplo de RPC/RMI): El cliente simplemente invoca una función como si fuera local:

```
lista_productos = obtenerInventarioActual();
```

- **Lo que el cliente ve:** Una llamada a una función (o método, si usa RMI, que es la versión orientada a objetos de RPC).
- **Lo que el cliente NO ve (la ubicación oculta):** El cliente no sabe que `obtenerInventarioActual()` no se ejecuta en su propia máquina, sino en un servidor remoto (por ejemplo, el servidor con la dirección IP `10.0.0.12`). El sistema de comunicación distribuida (RPC/RMI) se encarga de:

1. **Localizar** el procedimiento o el objeto remoto.
2. **Serializar** (transformar la estructura de datos en una secuencia de bytes para la transmisión) los parámetros.
3. **Transmitir** la solicitud a través de la red (usando protocolos como TCP/IP).
4. El resultado es que **la función se invoca como si fuera local**, ocultando el hecho de que el recurso (la función `obtenerInventarioActual`) está ubicado en otro nodo de la red.

Otro ejemplo de un mecanismo que busca esta abstracción son los **Servicios Web**, que permiten que otros sistemas invoquen funciones remotamente como si fueran locales.

Abstracción (El Principio General)

El principio general de la **Transparencia de Ubicación** es un subconjunto de la característica general de **Transparencia** en la Programación Distribuida.

La abstracción de este concepto es la siguiente:

El sistema de programación distribuida debe gestionar automáticamente la complejidad de encontrar y acceder a los recursos remotos, permitiendo que el desarrollador interactúe con ellos de manera uniforme, sin necesidad de conocer su dirección física o red.

Esto es crucial para:

1. **Mantenimiento y Evolución:** Si el recurso (como un servicio web) se mueve a un servidor diferente o se replica en múltiples nodos, el código del cliente que accede a él no necesita ser modificado.
2. **Escalabilidad:** Contribuye a la **escalabilidad** del sistema, ya que los nodos pueden aumentar y los recursos pueden reubicarse eficientemente

sin romper la comunicación del cliente.

3. **Uniformidad:** Se alinea con la Transparencia de Acceso, donde la forma de acceder a los recursos es uniforme, sin importar su ubicación local o remota

▼ Transparencia de acceso:

Ejemplo Concreto: Mecanismos de Alto Nivel

Los mecanismos de comunicación de alto nivel están diseñados específicamente para proporcionar esta uniformidad de acceso:

A. RPC y RMI

La principal manera de lograr la Transparencia de Acceso es a través de mecanismos que simulan la ejecución local de código que en realidad es remoto:

- **Llamada a Procedimiento Remoto (RPC):** RPC permite que un programa invoque funciones o procedimientos que están ubicados en otra máquina, **como si fueran locales**. El programador simplemente llama a la función sin preocuparse por los detalles de la comunicación en red.
- **Invocación a Métodos Remotos (RMI):** Esta es la versión orientada a objetos del RPC, específica de Java, y también permite invocar métodos remotos **como si fueran locales**.

B. Servicios Web

Los Servicios Web también cumplen con el principio de acceso uniforme:

- Un servicio web actúa como una "puerta" de acceso a funciones o datos que ofrece una aplicación.
- Permiten que otros sistemas puedan **invocar funciones remotamente como si fueran locales**.
- Los servicios web facilitan la comunicación entre aplicaciones en distintos lenguajes o plataformas utilizando protocolos estándar como HTTP.

3. Contraste con la Falta de Transparencia (Bajo Nivel)

Cuando la transparencia de acceso no existe, el programador debe gestionar explícitamente el hecho de que el recurso es remoto:

- **Sockets:** Al usar *sockets*, el programador está obligado a gestionar detalles de bajo nivel, como el establecimiento de la conexión, el envío y la recepción de datos a través de la red. Esto requiere que el código diferencie entre el acceso local y el acceso remoto, rompiendo la uniformidad.

▼ Transparencia de concurrencia:

Ocultar la dificultad de la coordinación del sistema distribuido. Permite que el cliente o el programador interactúen con los recursos como si fueran los únicos usuarios en el sistema, sin tener que preocuparse por la sincronización, los bloqueos o la posible inconsistencia de datos que resulta de las operaciones simultáneas. El sistema distribuido se encarga de que las operaciones de los múltiples usuarios se procesen correctamente y sin interferencia mutua

La **concurrencia** es una característica crucial de la **programación distribuida** que permite la **ejecución simultánea** de múltiples procesos. Esto maximiza la utilización de los recursos y mejora los tiempos de respuesta.

Característica	Ejecución Secuencial	Ejecución Concurrente
Orden	Proceso se ejecuta uno tras otro	Procesos pueden ejecutarse al mismo tiempo
Utilización de Recursos	Utilización parcial	Utilización completa
Tiempos de Respuesta	Más largos	Más cortos

La ejecución concurrente se puede representar así:

Proceso 1 →
Proceso 2 →
Proceso 3 →

Cada proceso se ejecuta paralelamente, optimizando la eficiencia del sistema.

▼ Transparencia de replicación y fallos:

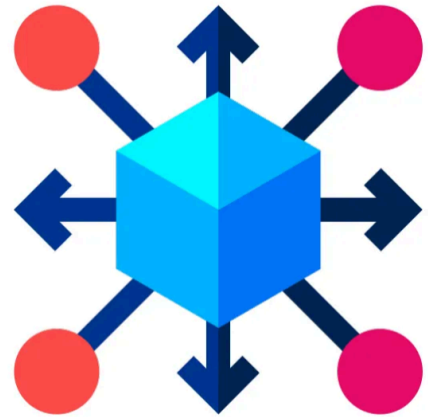
- **Replicación:** El cliente no necesita saber si los datos o servicios a los que accede están replicados en varios nodos para mejorar el rendimiento o la disponibilidad. El acceso es uniforme a pesar de la duplicación interna.

- **Fallos:** El sistema debe gestionar automáticamente los errores y fallos de los componentes (por ejemplo, si un nodo falla, otro toma su lugar) sin que el proceso del cliente se detenga o se dé cuenta del fallo ocurrido.

▼ Comunicación

Comunicación

- **Intercambio de mensajes:** base de la comunicación distribuida.
- **Modelos de interacción:**
 - **Cliente-servidor**
 - **Peer-to-peer:** Todos los nodos pueden actuar como clientes y servidores al mismo tiempo.
- **Mecanismos de comunicación:**
 - **RPC (Remote Procedure Call):** Permite que un programa invoque funciones o procedimientos ubicados en otra máquina, como si fueran locales.
 - **RMI (Remote Method Invocation):** Versión orientada a objetos del RPC, específica de Java.
 - **Sockets:** Permiten la comunicación a bajo nivel entre procesos a través de la red. Requieren que el programador gestione detalles como la conexión, envío y recepción de datos.
 - **Servicios Web:** Permiten la comunicación entre aplicaciones en distintos lenguajes o plataformas usando protocolos estándar (como HTTP).



Métodos de Comunicación	Características	Ejemplos
RPC (Remote Procedure Call)	Llama a funciones en otro componente	gRPC, SOAP
Message Queues	Envía y recibe mensajes	RabbitMQ, Kafka
Web APIs	Permite interacciones HTTP	RESTful APIs

Un ejemplo es una aplicación web que utiliza **Web APIs** para comunicarse con un **servidor**.

Cliente $\xrightarrow{\text{Solicitud HTTP}}$ Servidor

En este diagrama, el **cliente** envía una solicitud HTTP al **servidor**, permitiendo la ejecución y retorno de datos.

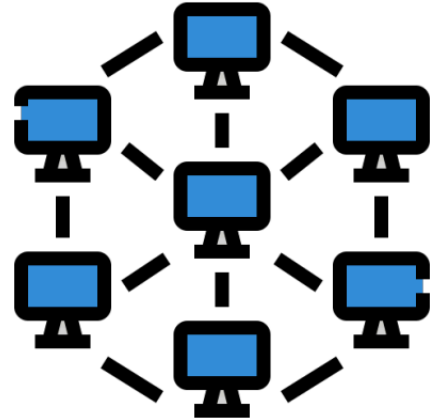
Servicios Web: Comunicación entre aplicaciones en distintos lenguajes o plataformas usando protocolos estándar como HTTP

▼ Conceptos básicos de Comunicación en Red

TCP/IP (*Transmission Control Protocol/Internet Protocol*)

- **IP (*Internet Protocol*)**

- IP es como si fuera el “sistema de direcciones de Internet”.
- Cada dispositivo conectado a una red tiene una dirección IP única (como la dirección de su casa).
- IP se encarga de que los paquetes de datos lleguen de un punto A a un punto B, sin preocuparse por el orden o la confiabilidad.
- Es como el servicio postal que entrega una carta, pero no se asegura de que llegue completa o en el orden correcto si envías varias.



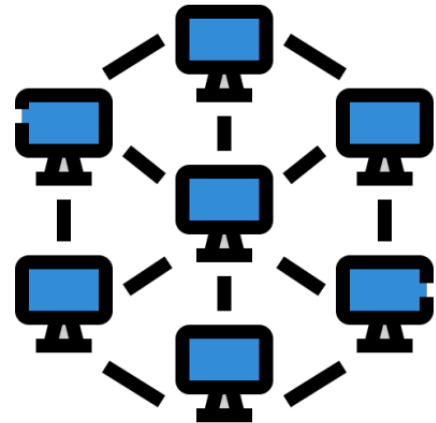
- **¿Para qué sirve?**

- Identificar el servidor y el cliente en la red.

TCP/IP (*Transmission Control Protocol/Internet Protocol*)

- **TCP (*Transmission Control Protocol*)**

- TCP es el “asegurador” de la comunicación.
- Trabaja sobre IP y garantiza que los paquetes de datos se entreguen de forma confiable, ordenada y sin errores.
- Establece una conexión entre dos puntos, la mantiene mientras dura la comunicación y la cierra cuando finaliza.
- Es como una conversación telefónica donde ambos interlocutores saben si el mensaje fue recibido y en qué orden.



- **¿Para qué nos sirve?**

- Los servicios web confían en TCP para una comunicación robusta y garantizada.
- Si un paquete de datos se pierde o llega desordenado, TCP se encarga de retransmitirlo o reordenarlo.

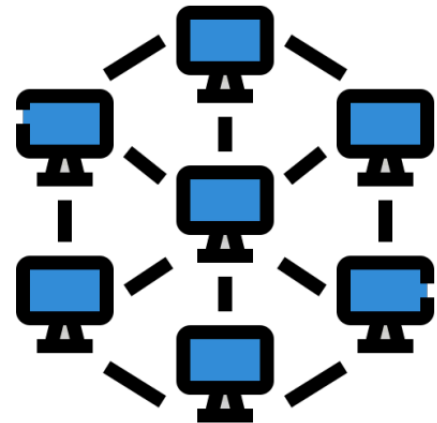
HTTP (*Hypertext Transfer Protocol*)

- **Definición**

- HTTP es un protocolo de aplicación (capa superior del modelo TCP/IP) para la transmisión de hipertexto.
- Permite a los navegadores web solicitar páginas web y a los servidores web enviarlas.

- **Modelo Petición-Respuesta:** HTTP funciona con un modelo simple:

- un cliente envía una petición a un servidor,
 - y el servidor envía una respuesta al cliente.
- Este es el protocolo sobre el que se construye gran parte de la web moderna, y es el pilar fundamental de los servicios web RESTful, aunque también es el protocolo de transporte subyacente para SOAP.



Serialización

- Es el proceso de convertir un objeto (o un grafo de objetos) en un formato que puede ser almacenado, transmitido o reconstruido más tarde.
- Básicamente, tomamos la estructura de datos en memoria y la transformamos en una secuencia lineal de bytes (o un formato de texto estructurado).
- **¿Por qué lo necesitamos?**
 - Para enviar datos a través de la red, debemos "aplanar" nuestros objetos en un formato que pueda ser transmitido bit a bit.



La **Serialización** es el proceso de "convertir un objeto (o un grafo de objetos) en un formato que puede ser almacenado, transmitido o reconstruido más tarde". Básicamente, transforma la estructura de datos en memoria en una secuencia lineal de bytes o un formato de texto estructurado.

Es crucial porque para enviar datos a través de la red, necesitamos "aplanar" nuestros objetos en un formato que pueda transmitirse bit a bit.

El proceso inverso es la **Deserialización**, que toma esa secuencia y la convierte de nuevo en un objeto funcional en el programa receptor. Los formatos de intercambio comunes son XML o JSON

What is UDDI?

- **UDDI** stands for **Universal Description, Discovery, and Integration**.
- It's a protocol for publishing and discovering web services.

What does a Service Registry do?

1. **Publishes** service descriptions
2. **Stores** service metadata
3. **Enables discovery** of services

Key components of UDDI:

- **BusinessEntity**: describes a business or organization offering services
- **BusinessService**: describes a service offered by a business
- **BindingTemplate**: provides technical details for accessing a service
- **tModel**: describes a service's technical interface

How does UDDI work?

- Services register with a **UDDI registry**
- Clients search the registry to **discover available services**
- Services provide **metadata** for discovery and integration

UDDI facilitates service discovery and integration in an SOA .

<https://desarrolloweb.com/articulos/que-es-rest-caracteristicas-sistemas.html>

[sesión 17 - Programación Distribuida.pdf](#)

[Gizmo](#) | [The easiest way to learn](#)

MODULAR

Perfecto  — tu `pom.xml` está bien estructurado, y para **desactivar la ejecución de los tests** en este módulo (`Persistence`), lo ideal es hacerlo dentro del bloque `<build>` agregando la configuración del plugin `maven-surefire-plugin`, que es el encargado de ejecutar los tests.

Te muestro exactamente cómo quedaría 

Versión modificada de tu `pom.xml` (con `skipTests`)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>pe.edu.pucp</groupId>
    <artifactId>SoftInv</artifactId>
    <version>1.0-SNAPSHOT</version>
  </parent>

  <artifactId>Persistence</artifactId>
  <packaging>jar</packaging>

  <dependencies>
    <dependency>
      <groupId>pe.edu.pucp</groupId>
      <artifactId>Model</artifactId>
      <version>1.0-SNAPSHOT</version>
      <type>jar</type>
    </dependency>
```

```

<dependency>
  <groupId>${project.groupId}</groupId>
  <artifactId>DBManager</artifactId>
  <version>${project.version}</version>
</dependency>

<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-params</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<properties>
  <maven.compiler.release>24</maven.compiler.release>
  <exec.mainClass>pe.edu.pucp.persistence.Persistence</exec.mainClass>
>
</properties>

<!--  Agrega esto →>
<build>
  <plugins>
    <!-- Desactiva ejecución de pruebas →>
    <plugin>

```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.1.2</version>
<configuration>
  <skipTests>true</skipTests>
</configuration>
</plugin>
</plugins>
</build>
<!--  Hasta aquí →
</project>
```

Qué hace esto

- Le dice a Maven: **"No ejecutes los tests de este módulo (Persistencia) al hacer `clean install` o `build`."**
- No afecta a otros módulos del proyecto.
- Si más adelante quieres volver a probar este módulo, **no necesitas editar el XML otra vez**: solo puedes forzarlo desde línea de comando:

```
mvn clean install -DskipTests=false
```

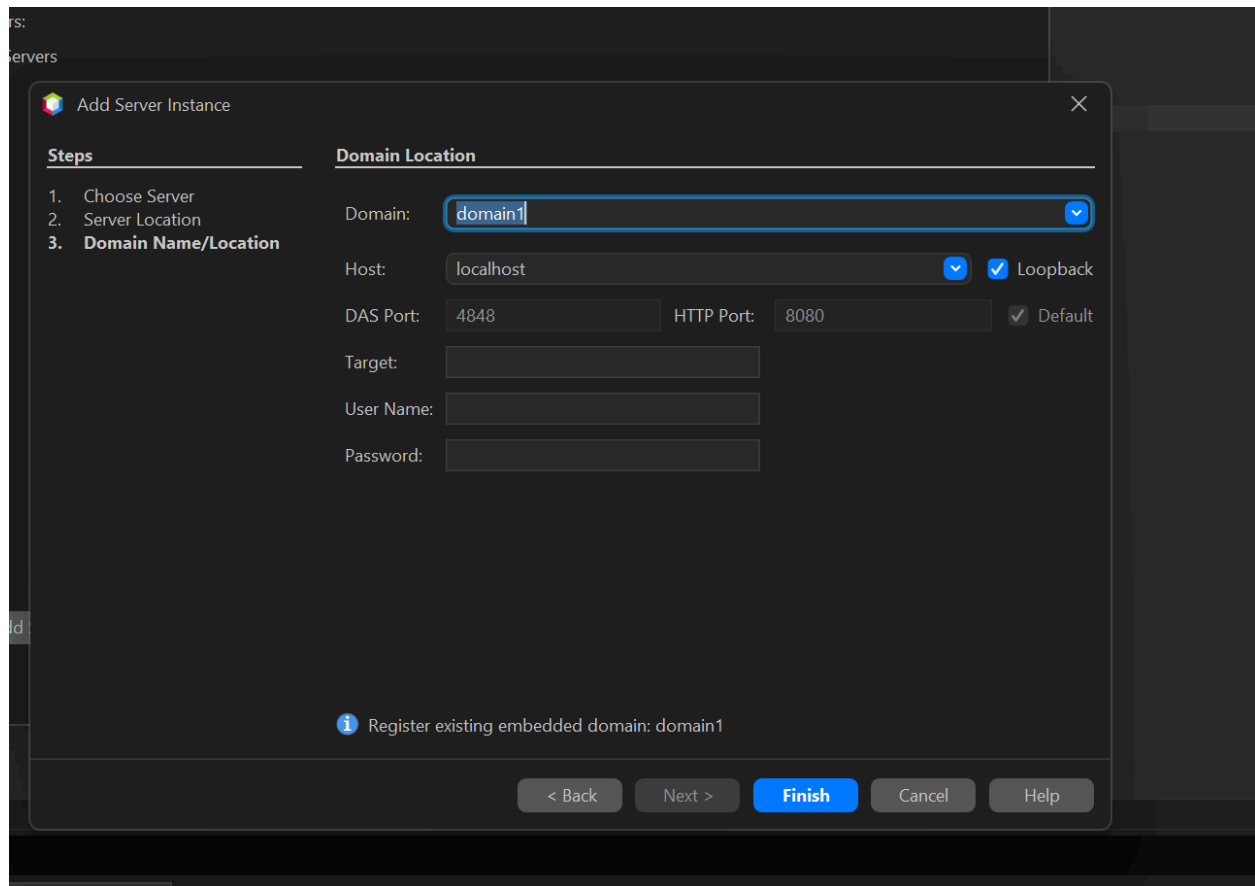
Esto sobrescribe temporalmente la configuración del `pom.xml`.

Si también quieres hacerlo en **Manager**

Haz exactamente lo mismo en su `pom.xml`, dentro del `<build>` de ese módulo.

¿Quieres que te muestre cómo se vería si controlamos el `skipTests` **desde el POM padre** para activar o desactivar todos los módulos a la vez (con una sola propiedad)? Es una forma más elegante si piensas alternar entre ejecutar y no ejecutar los tests.

SOAP - SERVER



dale finish nomas

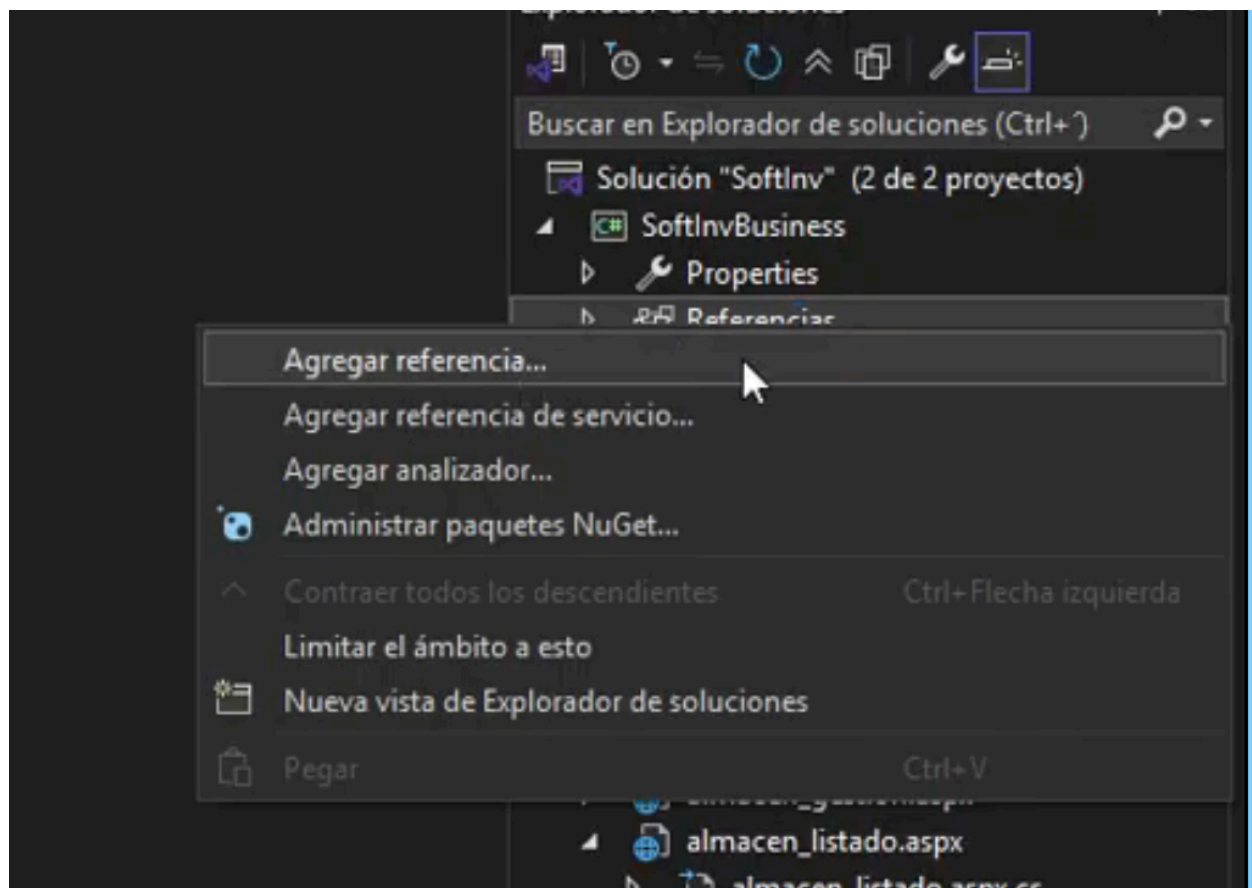
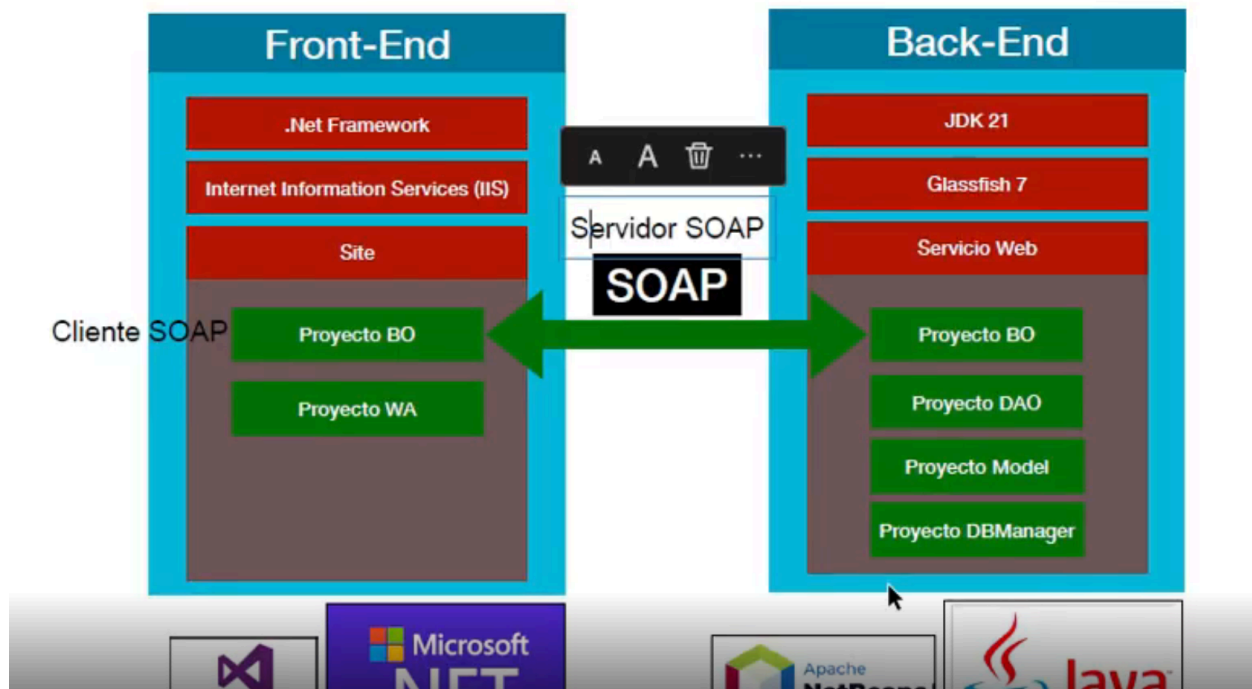
Lo he montado en mi ambiente de desarrollo para codear

cada que compile/ejecute → se ejecuta server web (glassfish): hace mas lento el desarrollo! Incorporas un elemento más

Para Mysql: se debe añadir otra cosa mas:

```
url = url.concat("?useSSL=false");
```

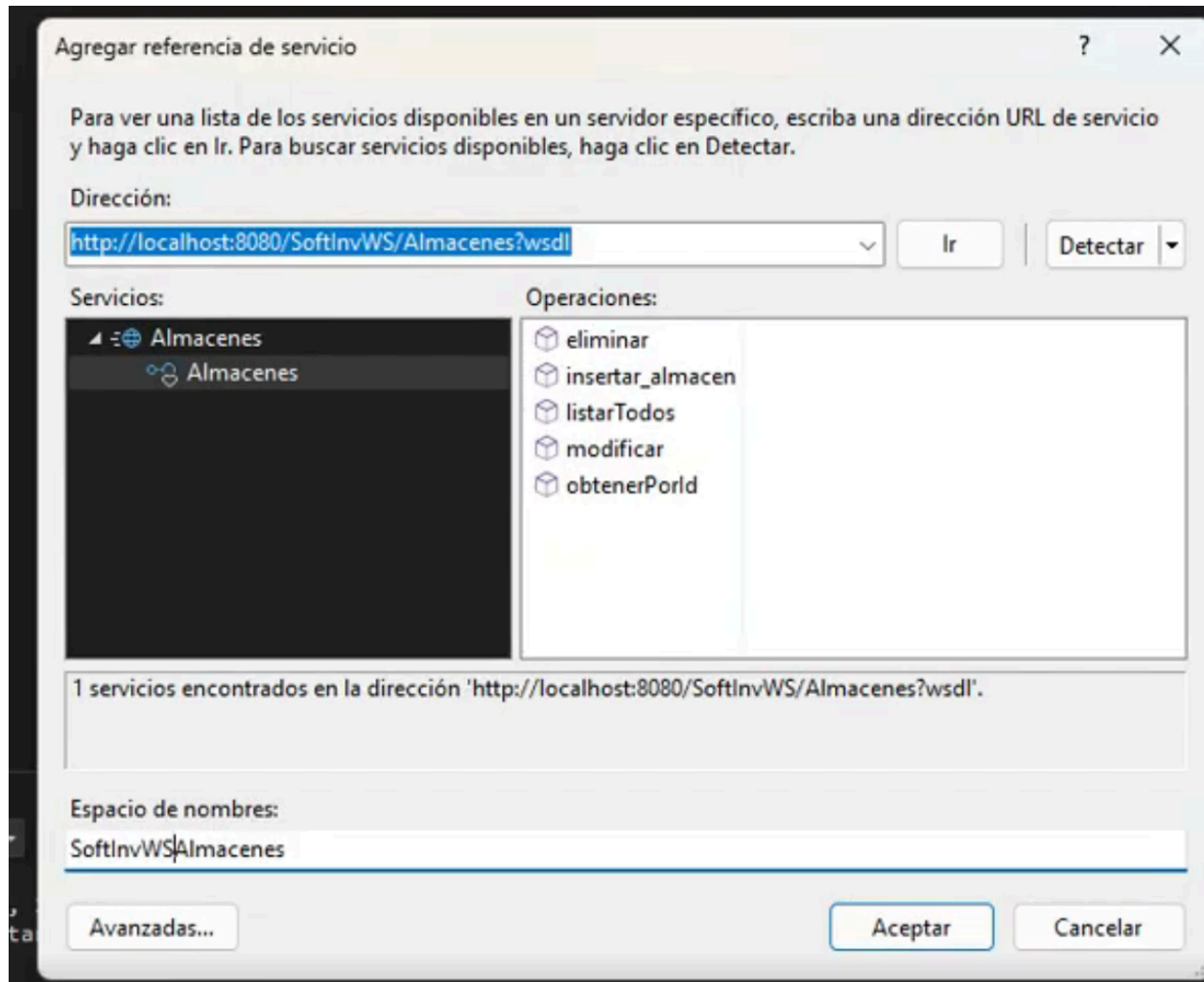
SOAP - CLIENTE



agregar referencia de servicio

¿Por que proxy soap pide como endpoint(lo que esta expuesto) al WSDL?

Porque contiene todos los metadatos(cual es el request, el response, los parametros, servicios)





IIS tambien te puede dar un servicio web

```

7 referencias
public class AlmacenBO
{
    private AlmacenesClient clienteSOAP;

    2 referencias
    public AlmacenBO()
    {
        this.clienteSOAP = new AlmacenesClient();
    }
}

```

```

-->
<configuration>
  <system.serviceModel>
    <bindings>
      <basicHttpBinding>
        <binding name="AlmacenesPortBinding" />
      </basicHttpBinding>
    </bindings>
    <client>
      <endpoint address="http://localhost:8080/SoftInvWS/Almac
        binding="basicHttpBinding" bindingConfiguration="Alm
        contract="SoftInvWSAlmacenes.Almacenes" name="Almace
      </client>
    </system.serviceModel>

```

ponerlo tanto en bo como wa

```
<configuration>
<system.serviceModel>
<bindings>
<basicHttpBinding>
<binding name="AlmacenesPortBinding"/>
</basicHttpBinding>
</bindings>
<client>
<endpoint address="http://localhost:8080/WS/Almacenes"
binding="basicHttpBinding" bindingConfiguration="AlmacenesPortBinding"
contract="SoftInvWSAlmacenes.Almacenes" name="AlmacenesPort"/>
</client>
</system.serviceModel>
</configuration>
```

¡Claro! Esta es una consulta clave. Cuando usas un servidor de aplicaciones como GlassFish, la forma estándar, más segura y eficiente de manejar conexiones a bases de datos **no es** usando `DriverManager` directamente en tu código (como en tu `DBManager` original).

En lugar de eso, configuras "Pools de Conexiones" en GlassFish, y tu aplicación simplemente "pide" una conexión de ese pool. Esto se hace a través de **JNDI (Java Naming and Directory Interface)**.

Aquí tienes los pasos más importantes, divididos en "Servidor" (lo que configuras en GlassFish) y "Cliente" (lo que modificas en tu aplicación Java).

Lado Servidor (Configuración en GlassFish)

Tu objetivo aquí es crear los "pools" de conexiones que tu aplicación usará. Esto se hace una sola vez desde la consola de administración de GlassFish (usualmente en `http://localhost:4848`).

1. Copiar los Drivers JDBC

GlassFish debe conocer los "drivers" de MySQL y MSSQL.

- Busca los archivos `.jar` de los drivers (ej. `mysql-connector-java-X.X.jar` y `mssql-jdbc-X.X.X.jreX.jar`).
- Cópialos en el directorio `lib` de tu dominio de GlassFish.
 - Por ejemplo: `[Ruta_GlassFish]/glassfish/domains/domain1/lib/`
- **Reinicia el servidor GlassFish** para que cargue los nuevos drivers.

2. Crear los Pools de Conexiones

Un "pool" es un conjunto de conexiones de base de datos listas para usar. Crearás uno para MySQL y otro para MSSQL.

1. Ve a la consola de admin (`http://localhost:4848`).
2. En el menú de la izquierda, ve a: **Resources** → **JDBC** → **JDBC Connection Pools**.
3. Haz clic en **New**.
4. **Para MySQL:**
 - **Pool Name:** `MySqlPool` (o el nombre que quieras).
 - **Resource Type:** `javax.sql.DataSource` .
 - **Database Driver Vendor:** `MySQL` .
 - Haz clic en **Next**.
 - Baja hasta **Additional Properties**.
 - Configura las propiedades: `ServerName` (o `Url`), `PortNumber`, `DatabaseName`, `User`, `Password` .
 - **Importante:** Borra la contraseña de tu archivo `jdbc.properties` . ¡Ahora vivirá de forma segura en GlassFish!
 - Haz clic en **Finish**.
5. **Prueba el Pool:** Selecciona el `MySqlPool` que acabas de crear y haz clic en **Ping**. Debería decir "Ping Succeeded".

6. Repite los pasos 3-5 para MSSQL (Pool Name: `MSSqlPool`, Driver Vendor: `Microsoft SQL Server`).

3. Crear los Recursos JDBC (JNDI)

Este es el "nombre público" que tu aplicación usará para encontrar el pool.

1. Ve a **Resources** → **JDBC** → **JDBC Resources**.
2. Haz clic en **New**.
3. **Para MySQL:**
 - **JNDI Name:** `jdbc/miBDMysql` (Este es el nombre clave).
 - **Pool Name:** Selecciona `MySqlPool` (el que creaste en el paso 2).
4. Haz clic en **OK**.
5. **Repite para MSSQL:**
 - **JNDI Name:** `jdbc/miBDMSSQL`
 - **Pool Name:** `MSSqlPool`

¡Listo! El servidor GlassFish ahora está sirviendo dos conexiones de base de datos en las "direcciones" JNDI `jdbc/miBDMysql` y `jdbc/miBDMSSQL`.



Lado Cliente (Tu Aplicación Java)

Ahora, en lugar de que tu `DBManager` lea un archivo `.properties` y use `DriverManager`, le enseñaremos a buscar los recursos JNDI que GlassFish provee. Esto hace tu código mucho más limpio y portable.

1. Mapear los Recursos (web.xml y glassfish-web.xml)

Tu aplicación debe declarar que va a usar esos recursos JNDI.

A. En `src/main/webapp/WEB-INF/web.xml`:

Añade esto para "declarar" los nombres que usarás dentro de tu código.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ...>
```

```

<resource-ref>
  <description>Conexion a BD MySQL</description>
  <res-ref-name>jdbc/mysqlDB</res-ref-name> <res-type>javax.sql.DataSource</res-type>
  <res-auth>Container</res-auth>
</resource-ref>

<resource-ref>
  <description>Conexion a BD MSSQL</description>
  <res-ref-name>jdbc/mssqlDB</res-ref-name> <res-type>javax.sql.DataSource</res-type>
  <res-auth>Container</res-auth>
</resource-ref>

</web-app>

```

B. En src/main/webapp/WEB-INF/glassfish-web.xml:

(Si no existe, créalo). Este archivo "mapea" los nombres de tu código (web.xml) con los nombres reales en GlassFish.

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE glassfish-web-app PUBLIC "-//GlassFish.org//DTD GlassFish Application Server 3.1 Servlet
3.0//EN" "http://glassfish.org/dtds/glassfish-web-app_3_0-1.dtd">
<glassfish-web-app>

  <resource-ref>
    <res-ref-name>jdbc/mysqlDB</res-ref-name> <jndi-name>jdbc/miBDMMySQL</jndi-name> </resource-ref>

  <resource-ref>
    <res-ref-name>jdbc/mssqlDB</res-ref-name> <jndi-name>jdbc/miBDMSSQL</jndi-name> </resource-ref>

</glassfish-web-app>

```

2. Refactorizar tu DBManager

Ahora simplificamos tu código Java. Ya no necesitas el `jdbc.properties` para esto, ni `Cifrado.descifrarMD5`.

DBManager.java (Modificado):

El DBManager abstracto ahora buscará un DataSource (el pool de conexiones) usando un nombre JNDI.

Java

```
package pe.edu.pucp.db;
```



```

import java.sql.Connection;
import java.sql.SQLException;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import javax.sql.DataSource;

public abstract class DBManager {

    private DataSource dataSource;

    // El constructor ahora busca el DataSource JNDI
    protected DBManager(String jndiName) {
        try {
            Context ctx = new InitialContext();
            // El prefijo "java:comp/env/" es estándar de JNDI
            // y se combina con el "res-ref-name" de web.xml
            this.dataSource = (DataSource) ctx.lookup("java:comp/env/" + jndiName);
        } catch (NamingException e) {
            // Error grave: si esto falla, la aplicación no puede funcionar
            throw new RuntimeException("No se pudo encontrar el JNDI DataSource: " + jndiName, e);
        }
    }

    // El método getConnection ahora es súper simple
    public Connection getConnection() throws SQLException {
        // Pide una conexión al pool de GlassFish
        return this.dataSource.getConnection();
    }

    // Este método abstracto sigue siendo vital
    public abstract String retornarSQLParaUltimoAutoGenerado();
}

```

DBManagerMySQL.java (Simplificado):

Solo necesita llamar al constructor del padre con el nombre JNDI correcto (el de web.xml).

Java

```

package pe.edu.pucp.db;

public class DBManagerMySQL extends DBManager {
    protected DBManagerMySQL() {
        // Este nombre debe coincidir con el <res-ref-name> en web.xml
        super("jdbc/mysqlDB");
    }

    // (El método getUrl() ya no es necesario)
}

```

```

@Override
public String retornarSQLParaUltimoAutoGenerado() {
    return "select @@last_insert_id as id";
}
}

```

DBManagerMSSQL.java (Simplificado):

Java

```

package pe.edu.pucp.db;

public class DBManagerMSSQL extends DBManager {
    protected DBManagerMSSQL() {
        // Este nombre debe coincidir con el <res-ref-name> en web.xml
        super("jdbc/mssqlDB");
    }

    // (El método getURL() ya no es necesario)

    @Override
    public String retornarSQLParaUltimoAutoGenerado() {
        return "select @@IDENTITY as id";
    }
}

```

3. ¡Usar tu GestorMultiDB !

La clase `GestorMultiDB` que te propuse en la respuesta anterior **funciona perfectamente y sin cambios** con este nuevo diseño.

Java

```

// Esta clase sigue igual y ahora usa el nuevo DBManager basado en JNDI
GestorMultiDB.getInstance().getMySQLConnection();
GestorMultiDB.getInstance().getMSSQLConnection();

```

Resumen de Beneficios

- **Seguridad:** Tus contraseñas ya no están en archivos `.properties` en tu código, sino gestionadas por GlassFish.
- **Rendimiento:** GlassFish gestiona el "pool" de conexiones, que es mucho más rápido que abrir y cerrar conexiones nuevas cada vez.
- **Portabilidad:** Puedes desplegar tu archivo `.war` en un entorno de producción (QA, Producción) y simplemente configurar los JNDI en ese servidor GlassFish

para que apunten a las bases de datos correctas (ej. `produccion_db` en lugar de `desarrollo_db`) **sin cambiar una sola línea de tu código Java.**

¿Te gustaría ver cómo quedaría la clase `GestorMultiDB` completa usando este nuevo enfoque?

¡Claro! Fue un gran proceso de depuración. Aquí tienes un resumen de los errores más comunes que resolvimos, por qué ocurrieron y cómo solucionarlos.

1. El `NullPointerException` (El más importante)

- **El Error (Síntoma):**

```
...because "this.conexion" is null
```

Ocurría al llamar a `.prepareCall()` o `.setAutoCommit()` .

- **El Problema Real:**

El código en tu capa de persistencia (DAO) intentaba usar una conexión a la base de datos, pero la variable `this.conexion` estaba null. Esto pasaba porque el Pool de Conexiones de GlassFish estaba fallando y no le entregaba una conexión válida a tu aplicación.

- **La Solución (La Clave):**

1. Ir a la consola de GlassFish (`http://localhost:4848`).
2. Navegar a **Resources** → **JDBC** → **JDBC Connection Pools**.
3. Seleccionar tu pool y presionar **"Ping"**.
4. Como el Ping fallaba, entramos a editar el pool y corregimos las **"Additional Properties"** (usuario, contraseña, URL de la base de datos) hasta que el "Ping" dio "Success".

2. El Error de Versión de Java

- **El Error (Síntoma):**

```
java.lang.UnsupportedClassVersionError: ... has unsupported major or minor version numbers
```

- El Problema Real:

Estabas compilando tu proyecto (tus .jar como Business.jar) con un JDK más nuevo (ej. JDK 24) del que tu servidor GlassFish estaba usando para ejecutar (ej. JDK 21).

- **La Solución:**

1. Abrir el `pom.xml` principal de tu proyecto.
2. Ajustar las propiedades de Maven para que la compilación coincidiera con el JDK de GlassFish (en tu caso, 21):XML

```
<properties>
<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>
```

3. Ejecutar `mvn clean install` para recompilar todo.

3. El Error de Puerto en Uso

- **El Error (Síntoma):**

```
java.net.BindException: Address already in use: bin (port 1527)
```

- El Problema Real:

El puerto 1527 (usado por la base de datos Java DB/Derby) ya estaba ocupado por otro proceso, probablemente una instancia anterior de NetBeans o GlassFish que no se cerró bien.

- **La Solución:**

1. Abrir CMD como Administrador.
2. Encontrar el proceso (PID) que usaba el puerto: `netstat -aon | findstr "1527"`
3. Matar ese proceso: `taskkill /F /PID <numero_del_PID>`

4. El Error de `mvn clean` (Archivo Bloqueado)

- **El Error (Síntoma):**

```
Failed to delete D:\...\[algun_archivo].jar
```

- El Problema Real:

El servidor GlassFish estaba corriendo y usando el "deployment" local (target). Esto bloqueaba los archivos, impidiendo que mvn clean pudiera borrarlos.

- La Solución:

Simplemente Detener ("Stop") el servidor GlassFish antes de ejecutar mvn clean install.

5. El Warning de `tools.jar`

- El Error (Síntoma):

```
'systemPath' for com.sun:tools:jar must specify an absolute path but is ${tools.jar}
```

- El Problema Real:

Una dependencia antigua (webservices-rt:2.3) hacía referencia a tools.jar, un archivo que no existe en JDK modernos.

- La Solución:

Ignorarlo. Es solo una advertencia (WARNING). Como la dependencia está marcada con `<scope>provided</scope>`, GlassFish se encarga de proveer la librería correcta en tiempo de ejecución, y este aviso no afecta en nada.

Espero que este resumen te sea muy útil.

¿Te gustaría que repasemos cómo configurar un cliente (como un proyecto de prueba o una aplicación de escritorio) para consumir el WSDL de tu servicio web ahora que está funcionando?

RESTFUL

Progra 3 Restful:

Arquitectura HTTP

mapper para serializar

URL, se debe leer de un properties

Mismo problema, consulta con dos bases de datos pero ahora puede ser cualquiera (no solo Mysql y Mssql) que cambios se deben hacer para que soporte motores independientes de su tipo. Pool de conexiones

Para restful en lugar de darle a nuevo web service le damos a nueva clase

- Aprender anotaciones, se usa en SpringBoot

- Oye clase, ahora tu expondras un servicio restful, te pondre una marca(anotacion)

- @Path("Almacenes")

- tenemos que producir y consumir json

- dado que restful soporta otros media type, le vamos a poner de forma explicita que produce que consume este tipo de archivo

- Todo el soporte a restful sera a traves de anotaciones

```
@Path("Almacenes")
```

```
@Produces(MediaType.APPLICATION_JSON)
```

```
@Consumes(MediaType.APPLICATION_JSON)
```

```
@GET
```

```
public ArrayList<AlmacenDTO>listarTodos(){
```

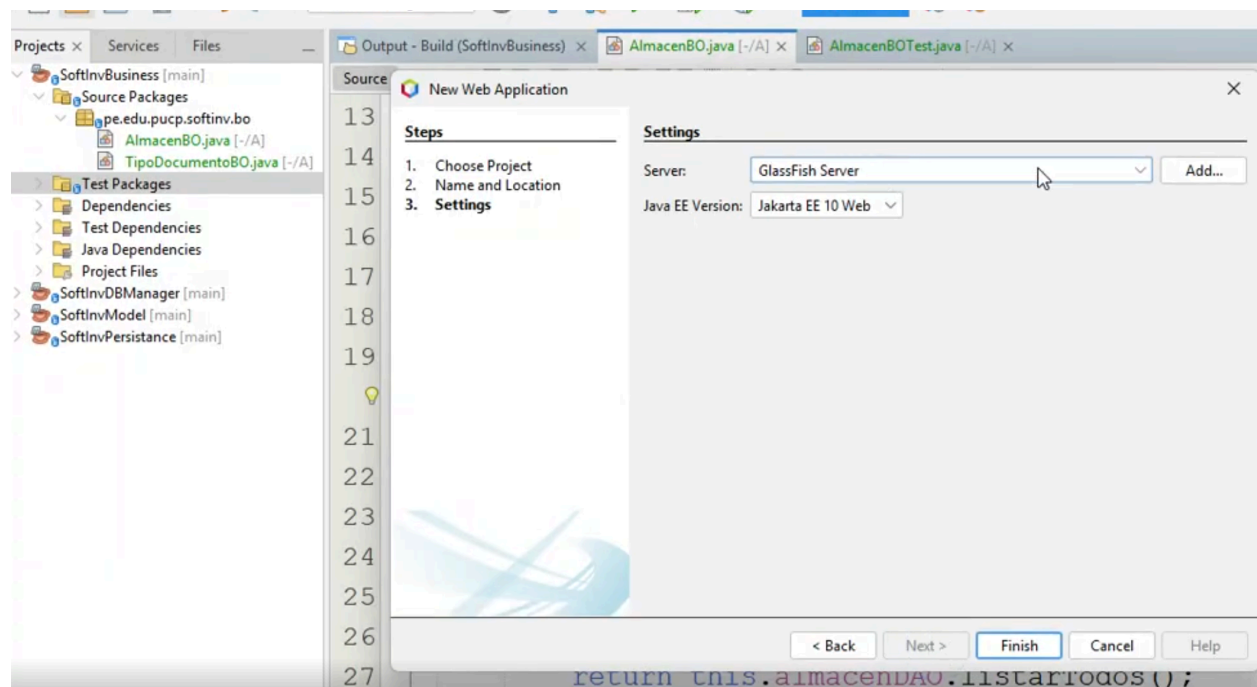
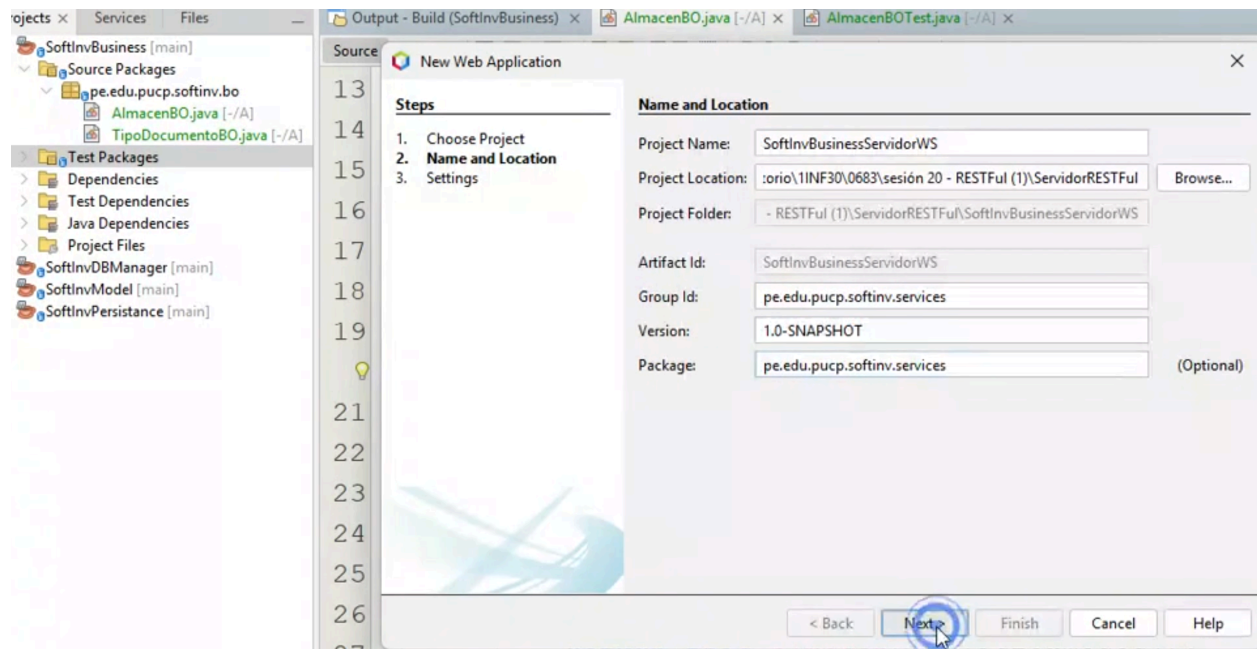
```
return this.almacenBO.listarTodos;
```

```
}
```

Se necesita cliente para restful, postman. Por que??

no entiendo bien las ventajas y diferencias entre restful y soap

,



10, jakarta 11 no funciona